



T.C. SAKARYA ÜNİVERSİTESİ
BİLGİSAYAR VE BİLİŞİM BİLİMLERİ FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ
YAPAY ZEKA DERSİ PROJE ÖDEVİ

G231210080 HANSIN EVREN TAŞEL

G221210020 AYHAN DALKIRAN

2. Öğretim C Grubu

<https://github.com/AyhanDalkiran/sau-bsm308-system-programming-project>

1. Giriş

Bu rapor, Sistem Programlama dersi kapsamında 2025-2026 Bahar döneminde geliştirilmiş olan 'tarsau' adlı arşivleme programının tasarım ve uygulama sürecini açıklamaktadır.

tarsau; tar, rar veya zip gibi popüler arşiv programlarına benzer biçimde çalışan, ancak herhangi bir sıkıştırma işlemi yapmayan, metin dosyası tabanlı bir arşivleme programıdır. Program, Linux (Unix) işletim sisteminde C dilinde yazılmış olup komut satırından iki temel modda kullanılabilir:

- Arşiv oluşturma modu (-b): Birden fazla ASCII metin dosyasını tek bir .sau arşiv dosyasında birleştirir.
- Arşiv açma modu (-a): Daha önce oluşturulmuş bir .sau arşivindeki dosyaları belirtilen dizine geri yükler.

Projenin temel amacı; dosya I/O işlemleri, ikili dosya formatı tasarımı, komut satırı argüman ayrıştırma ve POSIX dosya izinleri gibi sistem programlama konularını pratikte uygulamaktır.

2. Program Tasarımı

2.1. Komut Satırı Arayüzü

Program iki farklı modda çalışmaktadır:

```
tarsau -b [girdi_dosyasi...] -o [cikti_dosyasi]
tarsau -a [arsiv_dosyasi] [cikti_dizini]
```

Arşiv dosya adı belirtilmezse (-o parametresi kullanılmazsa) varsayılan olarak 'a.sau' ismi kullanılır. Giriş dosyası sayısı en fazla 32, toplam boyutu ise en fazla 200 MB olabilir.

2.2. .sau Arşiv Dosyası Formatı

.sau dosyası iki ana bölümden oluşmaktadır:

Bölüm	Açıklama
İlk 10 Bayt	Organizasyon bölümünün byte cinsinden boyutunu ASCII formatında tutar (sabit genişlik).
Organizasyon Bölümü	Her dosya kaydı ' ' ile ayrılır. Kayıt alanları: dosyaadı\0, izinler (4 oktal basamak), boyut (bayt).
Dosya İçerikleri	ASCII metin içerikleri herhangi bir ayırıcı olmadan art arda yazılır. Boyut bilgisi ayrıştırmayı sağlar.

Örnek Arşiv Dosyası Yapısı:

```
0000000049|ozel_test1.txt\0,0600,14|ozel_test2.txt\0,0755,13|Birinci dosya
İkinci dosya
```

İlk 10 bayt (0000000049), organizasyon bölümünün 49 bayt uzunluğunda olduğunu belirtir. Ardından gelen kayıtlar '|' ayırıcılarla birbirinden ayrılır. Dosya içerikleri ise organizasyon bölümünün hemen ardından bitleştirilerek yazılır.

3. Kod Yapısı ve Uygulama Detayları

3.1. Genel Yapı

Program tek bir kaynak dosyadan (src/main.c) oluşmaktadır. Temel fonksiyonlar şunlardır:

Fonksiyon	Görevi
<code>main()</code>	Argüman sayısını kontrol eder, -b veya -a modunu seçer.
<code>build_archive()</code>	Dosyaları okur, doğrular ve .sau arşivini oluşturur.
<code>extract_archive()</code>	.sau arşivini ayrıştırır, dosyaları hedefe çıkarır.
<code>makedirs()</code>	Gerekirse hedef dizini özyinelemeli olarak oluşturur (mkdir -p).

3.2. Arşiv Oluşturma (-b Modu)

`build_archive()` fonksiyonu sırasıyla aşağıdaki adımları gerçekleştirir:

- Komut satırı argümanları ayrıştırılır; -o bayrağı bulunarak çıkış dosyası belirlenir.
- Her giriş dosyası için `stat()` çağrısıyla dosya meta bilgileri (boyut, izinler) elde edilir.
- Dosyanın düzenli bir dosya (S_ISREG) olup olmadığı kontrol edilir.
- Toplam boyutun 200 MB sınırını aşıp aşmadığı denetlenir.
- Çıkış dosyası açılır; önce 10 baytlık yer tutucu yazılır, ardından her dosya için organizasyon kaydı eklenir.
- Her giriş dosyasının içeriği bayt bayt okunur; ASCII dışı karakter ($c > 127$) tespit edildiğinde hata mesajı verilir ve program temiz şekilde sonlanır.
- Son adımda `fseek()` ile dosyanın başına geri dönülerek gerçek header boyutu yazılır.

Kritik Kod Parçacığı — ASCII Doğrulama:

```
int c;
while ((c = fgetc(in)) != EOF) {
    if (c < 0 || c > 127) {
        fprintf(stderr, "'%s' girdi dosyasinin formati uyumsuzdur!\n", name);
        fclose(in); fclose(out); remove(output_file);
        return EXIT_FAILURE;
    }
    fputc(c, out);
}
```

3.3. Arşiv Açma (-a Modu)

`extract_archive()` fonksiyonu şu adımları izler:

- Arşiv dosyası açılır; ilk 10 bayt okunarak organizasyon bölümünün boyutu belirlenir.
- Organizasyon bölümü bir tampon belleğe (heap) yüklenir ve kayıtlar karakter karakter ayrıştırılır.

- Her kayıttan dosya adı (\0 sonlandırmalı), oktal izin kodu ve dosya boyutu çıkarılır.
- Hedef dizin belirtilmişse makedirs() ile özyinelemeli olarak oluşturulur.
- Her dosyanın içeriği, kayıttaki boyut bilgisi kadar arşivden okunarak hedef konuma yazılır.
- chmod() çağrısıyla orijinal dosya izinleri geri yüklenir.

Kritik Kod Parçacığı — İzin Geri Yükleme:

```
/* Restore original permissions */
if (chmod(outpath, records[i].perm) != 0) {
    fprintf(stderr, "Uyari: '%s' izinleri ayarlanamadi: %s\n",
        outpath, strerror(errno));
    /* Non-fatal: keep going */
}
```

4. Derleme ve Çalıştırma

4.1. Makefile

Proje, aşağıdaki Makefile ile tek komutla derlenebilmektedir:

```
CC      := gcc
CFLAGS := -std=c11 -Wall -Wextra -Werror -pedantic -pedantic-errors

all: clean | $(TARGET)

$(TARGET): $(OBJS)
    $(CC) $(CFLAGS) -o $@ $^

$(OBJ_DIR)/%.o: $(SRC_DIR)/%.c
    $(CC) $(CFLAGS) -I$(INC_DIR) -c $< -o $@
```

Derleme için aşağıdaki komut yeterlidir:

```
make
```

Derleme sonucunda bin/tarsau çalıştırılabilir dosyası oluşturulur.

4.2. Klasör Yapısı

```
proje/
├── src/
│   └── main.c          # Tüm kaynak kod
├── inc/                # Başlık dosyaları (şu an boş)
├── obj/                # Derlenmiş .o dosyaları
├── bin/
│   └── tarsau          # Çalıştırılabilir
├── test/              # Test dosyaları ve örnek arşivler
├── makefile
├── readme.txt
└── LICENSE
```

5. Test Senaryoları ve Ekran Çıktıları

5.1. Varsayılan Arşiv Adı Testi

Arşiv dosya adı belirtilmediğinde programın varsayılan olarak 'a.sau' kullanması beklenir.

Komut:

```
cd test/  
../bin/tarsau -b test1.txt test2.txt
```

Beklenen Çıktı:

```
'a.sau' arsivi 2 dosyayla basariyla olusturuldu.
```

Sonuç:

Test klasöründe a.sau isimli dosya oluşturuldu. Test BAŞARILI.

5.2. Maksimum Dosya Sayısı Testi

Programa en fazla 32 giriş dosyası verilebilir. 33 dosya verildiğinde hata mesajı beklenir.

Komut:

```
touch sinir_testi{1..33}.txt  
../bin/tarsau -b sinir_testi*.txt -o sinir.sau  
rm sinir_testi*.txt
```

Beklenen Çıktı:

```
Hata: Cok fazla girdi dosyasi belirtildi (maksimum 32).
```

Sonuç:

33. dosyada program hata mesajı verdi ve sonlandı. Test BAŞARILI.

5.3. Hatalı Format (İkili Dosya) Testi

ASCII olmayan (ikili) içerik barındıran dosyalar arşivlenmemeli; hata mesajı verilmelidir.

Komut:

```
head -c 100 /dev/urandom > t7.dat  
../bin/tarsau -b test1.txt test2.txt t7.dat -o hata_testi.sau
```

Beklenen Çıktı:

```
't7.dat' girdi dosyasinin formati uyumsuzdur!
```

Sonuç:

Program ASCII dışı bayt tespit edip düzgün çıkış yaptı. Test BAŞARILI.

5.4. 200 MB Boyut Sınırı Testi

Toplam giriş boyutu 200 MB'ı aştığında program uygun hata mesajı vermelidir.

Komut:

```
yes "Test verisi" | head -c 225000000 > buyuk_dosya.txt  
../bin/tarsau -b buyuk_dosya.txt -o boyut_testi.sau
```

Beklenen Çıktı:

```
Hata: Toplam girdi dosyasi boyutu 200MB'i asiyor.
```

Sonuç:

Program boyut sınırı aşımını doğru tespit etti. Test BAŞARILI.

5.5. Hatalı Uzantı Kontrolü (-a Modu)

.sau uzantısı olmayan bir dosya arşiv olarak açılmaya çalışıldığında hata mesajı beklenir.

Komut:

```
../bin/tarsau -a ozel_test1.txt
```

Beklenen Çıktı:

```
Arsiv dosyasi uygunsuz veya bozuk!
```

Sonuç:

Program ikili olmayan dosyayı doğru biçimde reddetti. Test BAŞARILI.

5.6. Geçerli Dizine Çıkarma Testi

Dizin adı verilmediğinde arşiv, çalışılan dizine açılmalıdır.

Kurulum:

```
echo 'Birinci dosya' > ozel_test1.txt  
echo 'ikinci dosya' > ozel_test2.txt  
chmod 600 ozel_test1.txt  
chmod 755 ozel_test2.txt  
../bin/tarsau -b ozel_test1.txt ozel_test2.txt -o test_arsiv.sau  
rm ozel_test1.txt ozel_test2.txt
```

Test Komutu:

```
../bin/tarsau -a test_arsiv.sau
```

Beklenen Çıktı:

```
-> ./ozel_test1.txt  
-> ./ozel_test2.txt  
'test_arsiv.sau' arshivindeki 2 dosya '.' dizinine basariyla acildi.
```

Sonuç:

Dosyalar mevcut dizine başarıyla çıkarıldı. Test BAŞARILI.

5.7. Yeni Dizin Oluşturma ve İzin Koruma Testi

Var olmayan bir hedef dizin belirtildiğinde program dizini oluşturmalı ve dosya izinlerini (600/755) korumalıdır.

Komut:

```
../bin/tarsau -a test_arsiv.sau yeni_dizin
```

İzin Kontrolü:

```
ls -l yeni_dizin/
```

Beklenen ls -l Çıktısı:

```
-rw----- 1 user user 14 May 23 00:00 ozel_test1.txt  
-rwxr-xr-x 1 user user 13 May 23 00:00 ozel_test2.txt
```

Sonuç:

yeni_dizin oluşturuldu; ozel_test1.txt 600, ozel_test2.txt 755 izinleriyle geri yüklendi. Test BAŞARILI.

6. Hata Yönetimi

Program, tüm olası hata durumlarını kontrol altında tutar; herhangi bir hatada kısmi çıktı dosyaları silinerek temiz biçimde çıkış yapılır. Temel hata senaryoları ve mesajları aşağıda verilmiştir:

Hata Durumu	Program Yanıtı
32'den fazla giriş dosyası	Hata mesajı, EXIT_FAILURE ile çıkış
Toplam boyut > 200 MB	Hata mesajı, EXIT_FAILURE ile çıkış
ASCII dışı bayt içeren dosya	'%s' girdi dosyasının formatı uyumsuzdur!
Geçersiz/bozuk arşiv dosyası	Arşiv dosyası uygunsuz veya bozuk!
Dosya açılmıyor (izin yok vb.)	strerror(errno) ile açıklayıcı mesaj
Yazma hatası	Kısmi dosya silindi, hata mesajı
chmod başarısız (ölümcül değil)	Uyarı mesajı, çalışmaya devam

8. Sonuç

Bu proje kapsamında, POSIX uyumlu bir arşivleme programı olan tarsau başarıyla geliştirilmiş ve test edilmiştir. Program, proje belgelerinde belirtilen tüm gereksinimleri karşılamaktadır:

- ASCII metin dosyalarını .sau formatında arşivleme
- Arşivlerin istenilen dizine doğru şekilde açılması
- Dosya izinlerinin korunması (POSIX chmod)
- 32 dosya ve 200 MB sınırlarının denetlenmesi
- Tüm hata durumlarında temiz çıkış ve anlamlı mesajlar
- make ile tek komutla derleme

Proje, sistem programlamanın temel kavramlarını — düşük seviyeli dosya I/O, ikili dosya formatı tasarımı ve POSIX API kullanımı — uygulamalı olarak pekiştirmiştir.